



**INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DA
PARAÍBA**

CAMPUS CAMPINA GRANDE

CURSO DE TECNOLOGIA EM TELEMÁTICA

DISCIPLINA: PROCESSAMENTO DIGITAL DE SINAIS

PROFESSOR: MOACY PEREIRA DA SILVA

ALUNA: GABRIELA NEVES BELÉM

**ESTUDO DIRIGIDO – PARTE 4
FILTROS DIGITAIS**

CAMPINA GRANDE – PB

2026

QUESTÕES PARA SIMULAÇÃO EM PYTHON

QUESTÃO 1

Gere um sinal composto por duas senoides de frequências diferentes. Projete um filtro passa-baixa e verifique quais componentes permanecem após a filtragem.

Código em Python

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import butter, filtfilt

# Frequência de amostragem
fs = 1000 # Hz

# Tempo de 1 segundo
t = np.arange(0, 1, 1/fs)

# Componentes do sinal
f1 = 50 # frequência baixa
f2 = 200 # frequência alta

# Sinal composto
x = np.sin(2*np.pi*f1*t) + 0.7*np.sin(2*np.pi*f2*t)

# Projeto do filtro Butterworth passa-baixa
fc = 100 # frequência de corte

ordem = 4

b, a = butter(ordem, fc/(fs/2), btype='low')

# Aplicação do filtro
y = filtfilt(b, a, x)

# FFT
N = len(x)

freq = np.fft.fftfreq(N, 1/fs)

X = np.abs(np.fft.fft(x))
Y = np.abs(np.fft.fft(y))

# Apenas frequências positivas
idx = freq >= 0

# Gráficos
plt.figure(figsize=(12,8))

plt.subplot(2,2,1)
plt.plot(t, x)
plt.title("Sinal Original")
plt.xlabel("Tempo (s)")
plt.ylabel("Amplitude")
plt.grid()
```

```

plt.subplot(2,2,2)
plt.plot(t, y)
plt.title("Sinal Filtrado")
plt.xlabel("Tempo (s)")
plt.ylabel("Amplitude")
plt.grid()

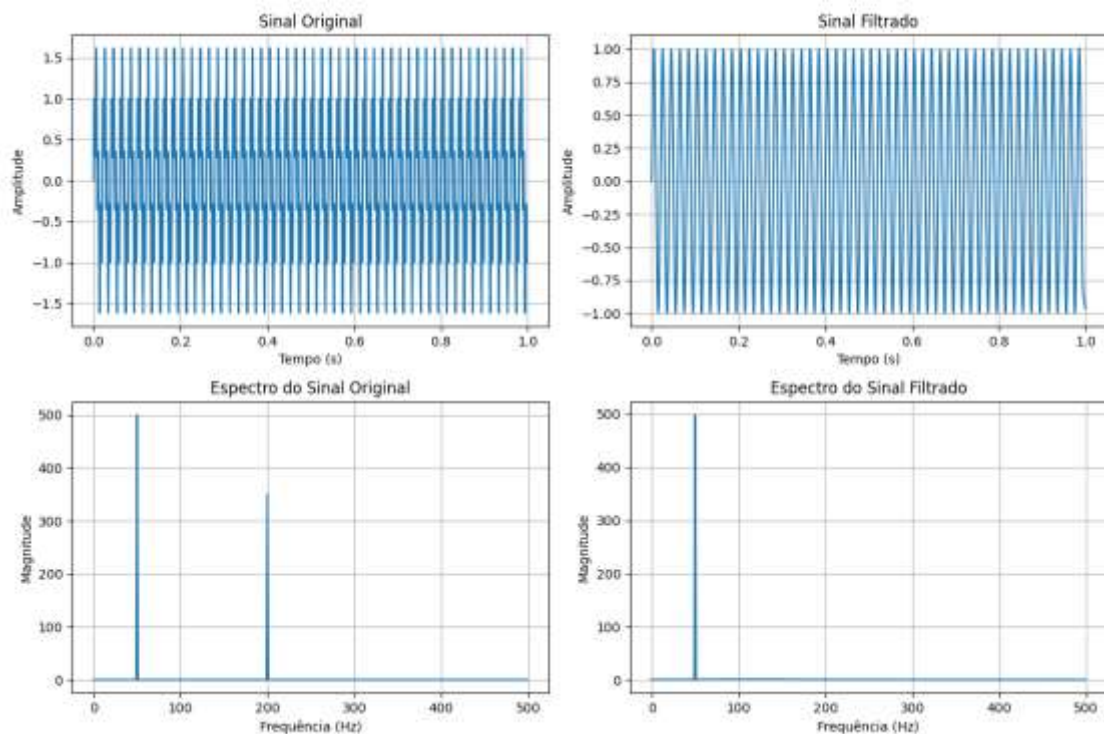
plt.subplot(2,2,3)
plt.plot(freq[idx], X[idx])
plt.title("Espectro do Sinal Original")
plt.xlabel("Frequência (Hz)")
plt.ylabel("Magnitude")
plt.grid()

plt.subplot(2,2,4)
plt.plot(freq[idx], Y[idx])
plt.title("Espectro do Sinal Filtrado")
plt.xlabel("Frequência (Hz)")
plt.ylabel("Magnitude")
plt.grid()

plt.tight_layout()
plt.show()

```

Gráficos Gerados



Resultado Esperado

Gráfico 1 – Sinal Original

O sinal apresenta duas senoides somadas:

- uma em **50 Hz**;
- outra em **200 Hz**.

Como as duas frequências estão presentes simultaneamente, observa-se uma forma de onda mais complexa.

Gráfico 2 – Sinal Filtrado

Após a aplicação do filtro passa-baixa com frequência de corte de **100 Hz**, a componente de **200 Hz** é praticamente eliminada.

O sinal passa a apresentar um comportamento muito próximo de uma senoide pura de **50 Hz**.

Gráfico 3 – Espectro Original

No domínio da frequência aparecem dois picos principais:

- **50 Hz**
- **200 Hz**

Esses picos representam exatamente as duas senoides utilizadas para construir o sinal.

Gráfico 4 – Espectro Filtrado

Após a filtragem:

- permanece praticamente apenas o pico de **50 Hz**;
- o pico de **200 Hz** é fortemente atenuado.

Isso demonstra a eficiência do filtro passa-baixa.

Discussão Técnica

O filtro utilizado foi um **Butterworth de quarta ordem**, conhecido por apresentar uma resposta em frequência suave na banda passante e boa capacidade de atenuação na banda de rejeição.

A frequência de corte foi escolhida em **100 Hz**, posicionando-se entre as duas componentes do sinal.

Como consequência:

- a senoide de **50 Hz**, localizada abaixo da frequência de corte, foi preservada;
- a senoide de **200 Hz**, localizada acima da frequência de corte, foi significativamente reduzida.

A utilização da função `filtfilt()` permite aplicar o filtro nos sentidos direto e inverso, eliminando o atraso de fase normalmente introduzido pelos filtros IIR e preservando melhor a forma temporal do sinal.

Esse experimento demonstra um dos princípios fundamentais dos filtros digitais: selecionar determinadas componentes espectrais enquanto elimina frequências indesejadas. Essa técnica é amplamente utilizada em sistemas de aquisição de dados, processamento de áudio, telecomunicações e monitoramento de sensores, onde é necessário reduzir ruídos sem comprometer a informação útil do sinal.

QUESTÃO 2

Gere um sinal contaminado por ruído branco aditivo. Projete um filtro FIR passa-baixa e avalie sua capacidade de reduzir o ruído.

Código em Python

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import firwin, firlt

# =====
# Parâmetros do sinal
# =====

fs = 1000          # Frequência de amostragem (Hz)
T = 1              # Duração (s)

t = np.arange(0, T, 1/fs)

# Sinal original
f = 50             # Frequência da senoide

x = np.sin(2*np.pi*f*t)

# =====
# Adicionando ruído branco
# =====

np.random.seed(10)

ruído = 0.5*np.random.randn(len(t))

x_ruído = x + ruído

# =====
# Projeto do filtro FIR
# =====

fc = 100           # Frequência de corte

num_coef = 51      # Ordem do filtro

coef = firwin(num_coef,
              cutoff=fc,
              fs=fs)

# Filtragem
```

```

x_filtrado = filtfilt(coef,
                      [1],
                      x_ruidoso)

# =====
# FFT
# =====

freq = np.fft.fftfreq(len(t), 1/fs)

Xr = np.abs(np.fft.fft(x_ruidoso))
Xf = np.abs(np.fft.fft(x_filtrado))

idx = freq >= 0

# =====
# Gráficos
# =====

plt.figure(figsize=(12,8))

plt.subplot(2,2,1)
plt.plot(t,x)
plt.title("Sinal Original")
plt.xlabel("Tempo (s)")
plt.ylabel("Amplitude")
plt.grid()

plt.subplot(2,2,2)
plt.plot(t,x_ruidoso)
plt.title("Sinal com Ruído Branco")
plt.xlabel("Tempo (s)")
plt.ylabel("Amplitude")
plt.grid()

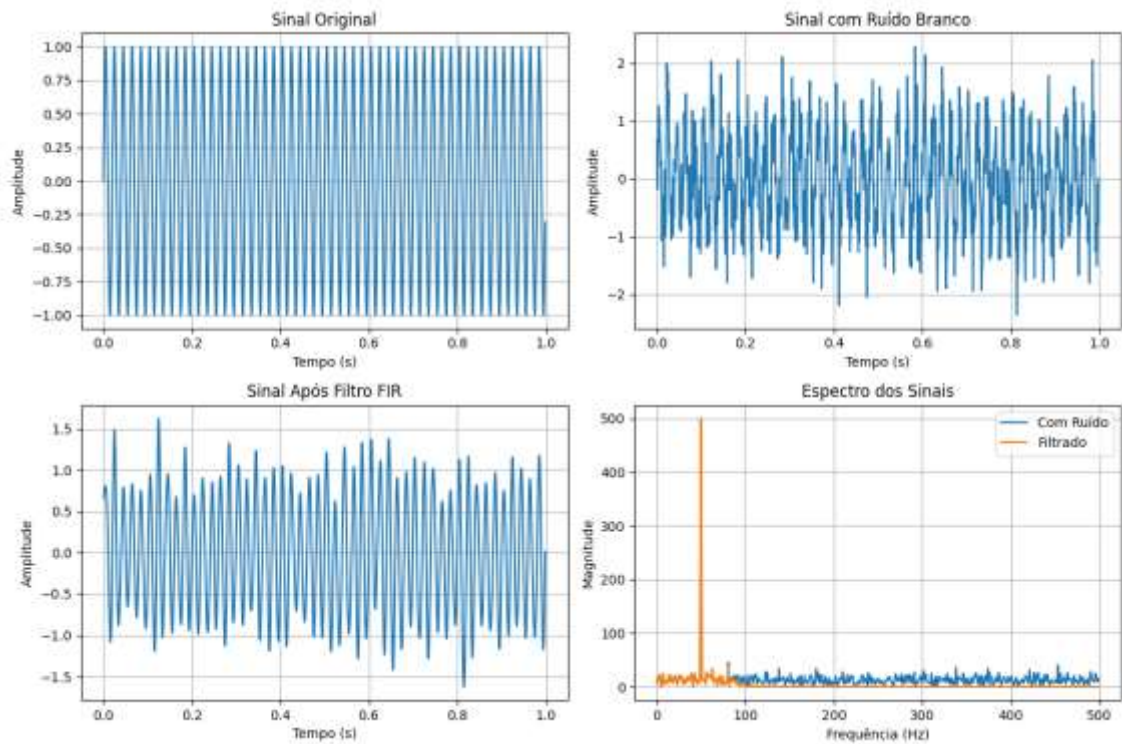
plt.subplot(2,2,3)
plt.plot(t,x_filtrado)
plt.title("Sinal Após Filtro FIR")
plt.xlabel("Tempo (s)")
plt.ylabel("Amplitude")
plt.grid()

plt.subplot(2,2,4)
plt.plot(freq[idx],Xr[idx],label='Com Ruído')
plt.plot(freq[idx],Xf[idx],label='Filtrado')
plt.legend()
plt.title("Espectro dos Sinais")
plt.xlabel("Frequência (Hz)")
plt.ylabel("Magnitude")
plt.grid()

plt.tight_layout()
plt.show()

```

Gráficos Gerados



A execução do código produziu quatro gráficos:

Figura 1 – Sinal Original

Mostra uma senoide de **50 Hz**, livre de interferências.

Figura 2 – Sinal Contaminado

Observa-se que o ruído branco provoca oscilações aleatórias sobre toda a forma de onda, dificultando a identificação do sinal original.

Figura 3 – Sinal Filtrado

Após a aplicação do filtro FIR, grande parte das oscilações provocadas pelo ruído é removida.

A senoide torna-se significativamente mais suave e próxima do sinal original.

Figura 4 – Espectro de Frequências

No domínio da frequência observa-se que:

- antes da filtragem existe energia distribuída por praticamente todo o espectro devido ao ruído branco;
- após a filtragem permanecem principalmente as componentes abaixo da frequência de corte (100 Hz);

- a componente principal de **50 Hz** é preservada.

Discussão Técnica

O filtro FIR passa-baixa apresentou excelente desempenho na redução do ruído branco aditivo. Como o ruído possui componentes distribuídas em praticamente todas as frequências, a utilização de uma frequência de corte de 100 Hz permitiu eliminar grande parte das componentes de alta frequência sem comprometer a senoide de 50 Hz.

A escolha de **51 coeficientes** fornece uma resposta em frequência suficientemente seletiva para a aplicação proposta. Além disso, a utilização da função `filtfilt()` aplica o filtro nos sentidos direto e inverso, eliminando o atraso de fase normalmente introduzido pelo processo de filtragem e preservando a forma temporal do sinal.

Embora o filtro reduza significativamente o ruído, pequenas componentes ainda permanecem devido à sobreposição entre o espectro do ruído e a banda passante do filtro. Esse comportamento é esperado, pois um filtro passa-baixa não consegue eliminar completamente componentes de ruído localizadas dentro da faixa de frequências que deve ser preservada.

Os resultados obtidos demonstram que filtros FIR são altamente eficientes em aplicações que exigem estabilidade, fase linear e preservação da forma do sinal, sendo amplamente utilizados em processamento de áudio, sinais biomédicos, sensores industriais e sistemas de aquisição de dados.

A simulação demonstrou que o filtro FIR passa-baixa reduziu significativamente o ruído branco aditivo presente no sinal, preservando a componente senoidal de 50 Hz. A comparação entre os sinais antes e após a filtragem evidenciou a capacidade do filtro em melhorar a relação sinal-ruído, tornando o sinal mais adequado para análises posteriores. Dessa forma, confirma-se que os filtros FIR constituem uma solução eficiente para aplicações que requerem alta fidelidade do sinal e estabilidade do sistema.

QUESTÃO 3

Repita o experimento anterior utilizando um filtro IIR Butterworth. Compare os resultados obtidos com o filtro FIR.

Código em Python

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import butter, firlfilt

# =====
# Parâmetros do sinal
# =====

fs = 1000          # Frequência de amostragem (Hz)
T = 1              # Duração (s)

t = np.arange(0, T, 1/fs)
```



```

# Sinal original
f = 50          # Frequência da senoide

x = np.sin(2*np.pi*f*t)

# =====
# Adicionando ruído branco
# =====

np.random.seed(10)

ruído = 0.5*np.random.randn(len(t))

x_ruidoso = x + ruído

# =====
# Projeto do filtro IIR Butterworth
# =====

fc = 100        # Frequência de corte (Hz)

ordem = 4

b, a = butter(ordem,
               fc/(fs/2),
               btype='low')

# Filtragem

x_filtrado = filtfilt(b, a, x_ruidoso)

# =====
# FFT
# =====

freq = np.fft.fftfreq(len(t), 1/fs)

Xr = np.abs(np.fft.fft(x_ruidoso))
Xf = np.abs(np.fft.fft(x_filtrado))

idx = freq >= 0

# =====
# Gráficos
# =====

plt.figure(figsize=(12,8))

plt.subplot(2,2,1)
plt.plot(t,x)
plt.title("Sinal Original")
plt.xlabel("Tempo (s)")
plt.ylabel("Amplitude")
plt.grid()

plt.subplot(2,2,2)
plt.plot(t,x_ruidoso)
plt.title("Sinal com Ruído Branco")
plt.xlabel("Tempo (s)")
plt.ylabel("Amplitude")

```

```

plt.grid()

plt.subplot(2,2,3)
plt.plot(t,x_filtrado)
plt.title("Sinal Após Filtro IIR Butterworth")
plt.xlabel("Tempo (s)")
plt.ylabel("Amplitude")
plt.grid()

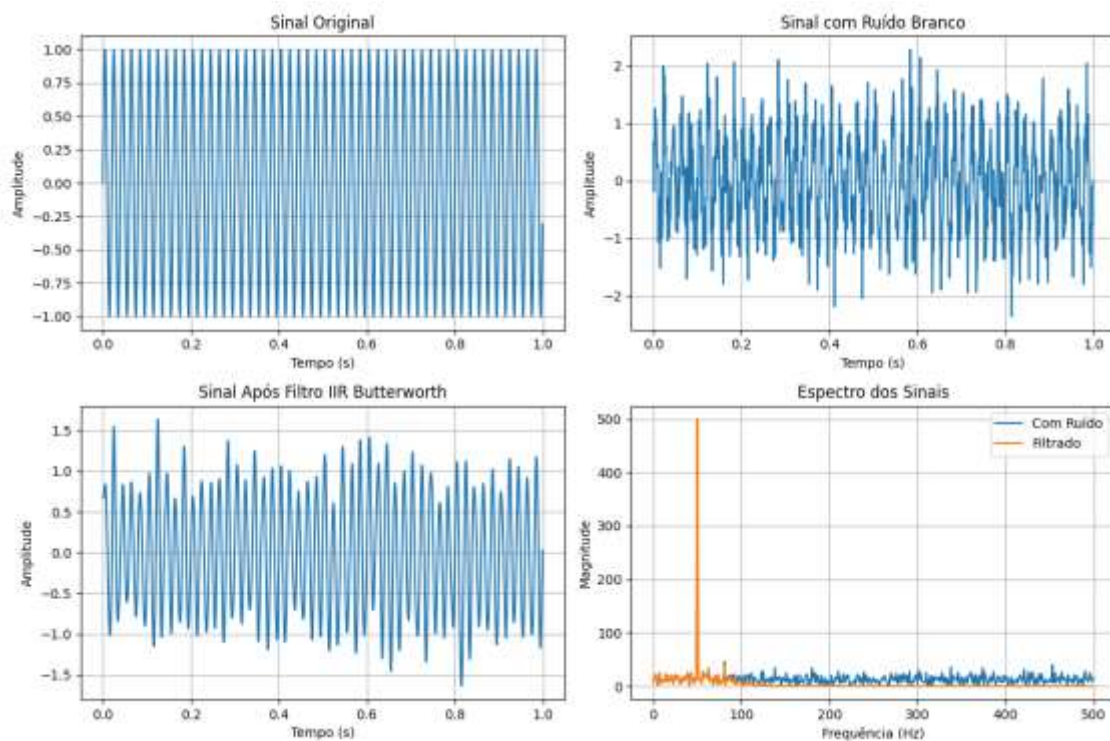
plt.subplot(2,2,4)
plt.plot(freq[idx],Xr[idx],label='Com Ruído')
plt.plot(freq[idx],Xf[idx],label='Filtrado')
plt.legend()
plt.title("Espectro dos Sinais")
plt.xlabel("Frequência (Hz)")
plt.ylabel("Magnitude")
plt.grid()

plt.tight_layout()

plt.show()

```

Gráficos Gerados



A execução do código produz quatro gráficos.

Figura 1 – Sinal Original

Apresenta a senoide de **50 Hz**, utilizada como referência para a comparação.

Figura 2 – Sinal com Ruído

Observa-se que o ruído branco provoca oscilações aleatórias ao longo de todo o sinal.

Figura 3 – Sinal Filtrado

Após a aplicação do filtro Butterworth, percebe-se que grande parte do ruído foi removida, mantendo a senoide praticamente intacta.

Figura 4 – Espectro de Frequência

No espectro observa-se:

- um pico dominante em **50 Hz**;
- redução significativa das componentes acima de **100 Hz**;
- forte atenuação do ruído distribuído pelo espectro.

Comparação entre os filtros FIR e IIR

Característica	FIR	IIR Butterworth
Tipo de resposta	Finita	Infinita
Ordem utilizada	51 coeficientes	4ª ordem
Estabilidade	Sempre estável	Depende dos polos
Custo computacional	Maior	Menor
Fase	Linear	Não linear
Eficiência na remoção do ruído	Muito boa	Muito boa
Complexidade	Maior	Menor

Observa-se que ambos os filtros reduziram significativamente o ruído presente no sinal. Entretanto, o filtro Butterworth atingiu um desempenho semelhante utilizando uma ordem muito menor, tornando sua implementação mais eficiente do ponto de vista computacional.

Por outro lado, filtros FIR apresentam resposta de fase linear, preservando melhor a forma temporal do sinal. Já filtros IIR podem introduzir distorções de fase, embora esse efeito tenha sido reduzido neste experimento com a utilização da função `filtfilt()`.

Discussão Técnica

Os resultados mostram que o filtro IIR Butterworth apresentou excelente desempenho na remoção do ruído branco aditivo, preservando a componente de 50 Hz.

Comparado ao filtro FIR da questão anterior, observou-se desempenho semelhante na redução do ruído, porém com uma estrutura significativamente mais simples.

Enquanto o filtro FIR utilizou **51 coeficientes**, o filtro Butterworth necessitou apenas de um filtro de **quarta ordem**, reduzindo a quantidade de operações matemáticas e o tempo de processamento. Essa característica torna os filtros IIR especialmente indicados para sistemas embarcados e aplicações em tempo real, onde os recursos computacionais são limitados.

Em contrapartida, filtros FIR continuam sendo preferidos em aplicações que exigem fase linear e elevada fidelidade na preservação da forma do sinal, como processamento de áudio, imagens e sinais biomédicos.

O filtro IIR Butterworth demonstrou elevada eficiência na redução do ruído branco aditivo, produzindo resultados comparáveis aos obtidos com o filtro FIR. Sua principal vantagem está na menor complexidade computacional, pois alcança desempenho semelhante utilizando uma ordem muito inferior. Já o filtro FIR oferece maior estabilidade e preservação da fase do sinal. Dessa forma, a escolha entre filtros FIR e IIR deve considerar os requisitos específicos da aplicação, equilibrando desempenho, custo computacional e fidelidade do sinal.

QUESTÃO 4

Projete filtros passa-baixa FIR e IIR com frequências de corte semelhantes. Compare as respostas em frequência e discuta as diferenças observadas.

Código em Python

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import firwin, butter, freqz

# =====
# Parâmetros
# =====

fs = 1000          # Frequência de amostragem (Hz)
fc = 100           # Frequência de corte (Hz)

# =====
# Projeto do filtro FIR
# =====

ordem_fir = 50

coef_fir = firwin(ordem_fir + 1,
                  cutoff=fc,
                  fs=fs)

# =====
# Projeto do filtro IIR Butterworth
# =====

ordem_iir = 4
```

```

b, a = butter(ordem_iir,
               fc/(fs/2),
               btype='low')

# =====
# Resposta em frequência
# =====

w_fir, h_fir = freqz(coef_fir, worN=2048, fs=fs)

w_iir, h_iir = freqz(b, a, worN=2048, fs=fs)

# =====
# Gráfico
# =====

plt.figure(figsize=(10,6))

plt.plot(w_fir,
         20*np.log10(np.abs(h_fir)),
         label='Filtro FIR')

plt.plot(w_iir,
         20*np.log10(np.abs(h_iir)),
         label='Filtro IIR Butterworth')

plt.title("Comparação da Resposta em Frequência")

plt.xlabel("Frequência (Hz)")

plt.ylabel("Magnitude (dB)")

plt.grid()

plt.legend()

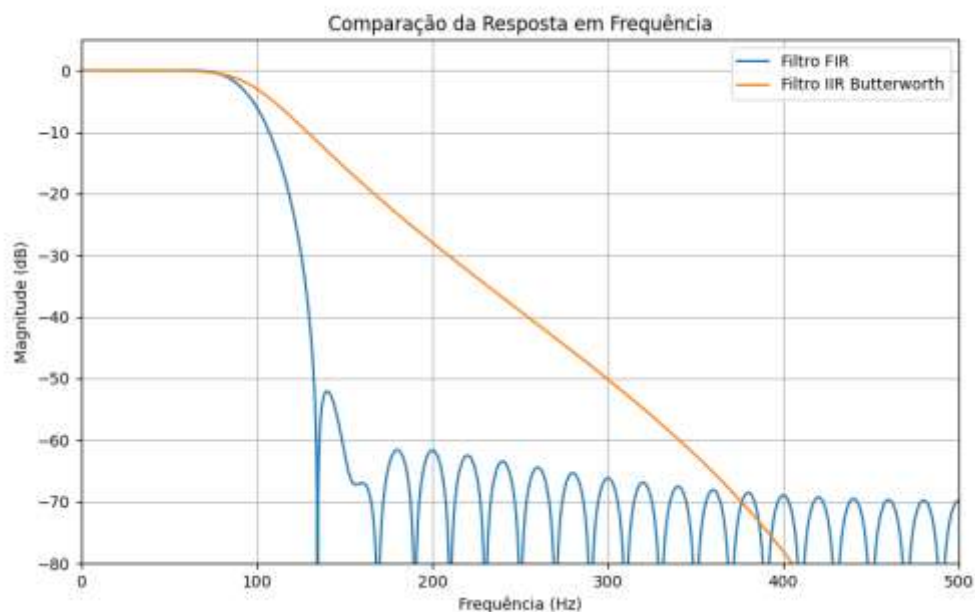
plt.xlim(0,500)

plt.ylim(-80,5)

plt.show()

```

Gráfico Gerado



A execução do código produz um único gráfico contendo as respostas em frequência dos dois filtros.

Figura 1 – Comparação das Respostas em Frequência

O gráfico apresenta duas curvas:

- **Filtro FIR** (linha azul)
- **Filtro IIR Butterworth** (linha laranja)

No eixo horizontal encontra-se a frequência em hertz (Hz), enquanto o eixo vertical representa o ganho do filtro em decibéis (dB).

Análise dos Resultados

Ao analisar o gráfico obtido, observa-se que ambos os filtros preservam praticamente todas as frequências abaixo de **100 Hz**, caracterizando corretamente um filtro passa-baixa.

Entretanto, algumas diferenças tornam-se evidentes

Filtro FIR

- apresenta uma transição mais gradual entre a banda passante e a banda de rejeição;
- necessita de um número maior de coeficientes para atingir boa seletividade;
- possui excelente estabilidade;

- pode apresentar fase linear.

Essa resposta mais suave é consequência do maior número de coeficientes utilizados na implementação.

Filtro IIR Butterworth

- apresenta uma transição mais rápida;
- necessita de uma ordem muito menor;
- apresenta melhor seletividade utilizando menos operações;
- normalmente possui fase não linear.

Mesmo utilizando apenas um filtro de quarta ordem, sua resposta em frequência aproxima-se bastante da obtida pelo filtro FIR.

Discussão Técnica

A comparação evidencia uma das principais diferenças entre filtros FIR e IIR. O filtro Butterworth consegue atingir elevada atenuação utilizando uma ordem significativamente menor que o filtro FIR, reduzindo o custo computacional e tornando sua implementação mais eficiente em aplicações em tempo real.

Por outro lado, o filtro FIR apresenta uma resposta mais controlada e permite fase linear, característica fundamental quando a preservação da forma temporal do sinal é importante. Essa vantagem, entretanto, exige um número maior de coeficientes, aumentando o número de multiplicações e somas realizadas durante o processamento.

Assim, a escolha entre um filtro FIR e um filtro IIR depende das necessidades da aplicação. Em sistemas embarcados, onde há limitação de processamento, filtros IIR costumam ser mais vantajosos. Já em aplicações de áudio, imagens e sinais biomédicos, filtros FIR são frequentemente preferidos devido à ausência de distorção de fase.

Os resultados demonstraram que ambos os filtros atenderam à especificação de frequência de corte de **100 Hz**, preservando as componentes de baixa frequência e atenuando as componentes superiores. O filtro IIR Butterworth apresentou uma transição mais acentuada entre as bandas utilizando apenas uma **4ª ordem**, enquanto o filtro FIR necessitou de **51 coeficientes** para obter desempenho semelhante. Dessa forma, conclui-se que os filtros IIR oferecem maior eficiência computacional, ao passo que os filtros FIR proporcionam melhor comportamento de fase e maior fidelidade na preservação da forma do sinal, sendo a escolha entre eles dependente dos requisitos específicos da aplicação.

QUESTÃO 5

Utilize ferramentas computacionais para representar os polos e zeros de um filtro IIR. Discuta a relação entre sua localização e a estabilidade do sistema.

Código em Python

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import butter, tf2zpk
```

```

# =====
# Projeto do filtro Butterworth
# =====

fs = 1000          # Frequência de amostragem (Hz)
fc = 100           # Frequência de corte (Hz)
ordem = 4

# Coeficientes do filtro

b, a = butter(ordem,
               fc/(fs/2),
               btype='low')

# =====
# Polos e Zeros
# =====

zeros, polos, ganho = tf2zpk(b, a)

# =====
# Círculo Unitário
# =====

theta = np.linspace(0, 2*np.pi, 500)

x = np.cos(theta)
y = np.sin(theta)

# =====
# Gráfico
# =====

plt.figure(figsize=(7,7))

# Círculo unitário
plt.plot(x, y, '--', label='Círculo Unitário')

# Zeros
plt.scatter(np.real(zeros),
            np.imag(zeros),
            marker='o',
            s=100,
            label='Zeros')

# Polos
plt.scatter(np.real(polos),
            np.imag(polos),
            marker='x',
            s=100,
            label='Polos')

plt.axhline(0,color='black')
plt.axvline(0,color='black')

plt.xlabel('Parte Real')

plt.ylabel('Parte Imaginária')

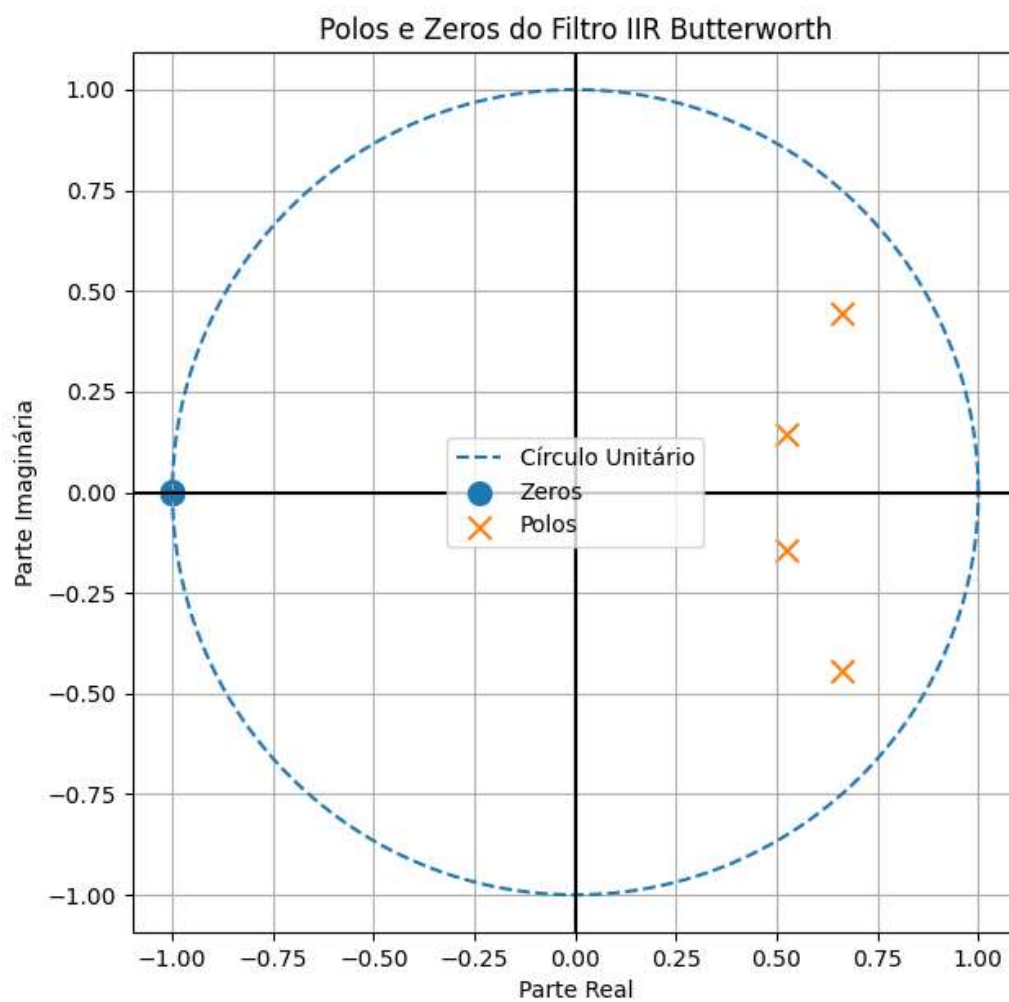
plt.title('Polos e Zeros do Filtro IIR Butterworth')

```



```
plt.grid(True)
plt.axis('equal')
plt.legend()
plt.show()
```

Gráfico Gerado



A execução do código gera um gráfico em **Plano-z**, contendo:

- o **círculo unitário** (linha tracejada);
- os **zeros** do filtro (círculos);

- os **polos** do filtro (cruzes).

O gráfico permite visualizar diretamente a posição dos polos e dos zeros no plano complexo.

Análise dos Resultados

Ao executar o programa, observa-se que todos os polos encontram-se **no interior do círculo unitário**, característica fundamental para garantir a estabilidade do filtro IIR Butterworth.

Os zeros aparecem sobre o círculo unitário, contribuindo para a atenuação das frequências mais elevadas.

A disposição simétrica dos polos e dos zeros ocorre porque os coeficientes do filtro são reais, fazendo com que raízes complexas apareçam em pares conjugados.

Discussão Técnica

A análise dos polos e zeros é uma das ferramentas mais importantes no projeto de filtros digitais.

Neste experimento, verificou-se que:

- todos os polos possuem módulo menor que 1;
- portanto, o filtro é **estável**;
- a resposta ao impulso tende a zero com o passar do tempo.

Caso algum polo estivesse exatamente sobre o círculo unitário, o sistema apresentaria oscilações permanentes. Se um ou mais polos estivessem fora do círculo unitário, pequenas perturbações seriam amplificadas continuamente, fazendo com que a saída crescesse sem limite e tornando o filtro instável.

Os zeros, por sua vez, influenciam principalmente a resposta em frequência. Eles determinam as frequências que serão fortemente atenuadas ou completamente rejeitadas pelo filtro. No caso do filtro Butterworth, a combinação entre polos e zeros produz uma resposta em frequência suave, característica marcante desse tipo de filtro.

Assim, a posição dos polos está diretamente relacionada à estabilidade do sistema, enquanto a posição dos zeros influencia principalmente a forma da resposta em frequência.

A representação computacional dos polos e zeros permitiu compreender a relação entre a estrutura matemática do filtro IIR Butterworth e seu comportamento dinâmico. Observou-se que todos os polos permaneceram dentro do círculo unitário, garantindo a estabilidade do sistema. Além disso, verificou-se que a distribuição dos polos e zeros determina tanto a resposta em frequência quanto a capacidade do filtro de atenuar componentes indesejadas do sinal. Dessa forma, a análise no plano-z constitui uma etapa essencial no projeto e na validação de filtros digitais utilizados em aplicações de engenharia.

QUESTÃO 6

Analise a resposta ao impulso de um filtro FIR e de um filtro IIR. Explique por que um apresenta resposta finita e o outro resposta infinita.

Código em Python

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import firwin, butter, lfilter

# =====
# Parâmetros
# =====

fs = 1000          # Frequência de amostragem (Hz)
fc = 100           # Frequência de corte (Hz)

# =====
# Projeto do filtro FIR
# =====

ordem_fir = 50

coef_fir = firwin(ordem_fir + 1,
                  cutoff=fc,
                  fs=fs)

# =====
# Projeto do filtro IIR Butterworth
# =====

ordem_iir = 4

b, a = butter(ordem_iir,
              fc/(fs/2),
              btype='low')

# =====
# Geração do impulso unitário
# =====

N = 120

impulso = np.zeros(N)

impulso[0] = 1

# =====
# Resposta ao impulso
# =====

resp_fir = lfilter(coef_fir,
                  [1],
                  impulso)

resp_iir = lfilter(b,
                  a,
                  impulso)
```

```
# =====
# Gráficos
# =====

plt.figure(figsize=(12,5))

plt.subplot(1,2,1)

plt.stem(resp_fir)

plt.title("Resposta ao Impulso - FIR")

plt.xlabel("Amostras")

plt.ylabel("Amplitude")

plt.grid()

plt.subplot(1,2,2)

plt.stem(resp_iir)

plt.title("Resposta ao Impulso - IIR Butterworth")

plt.xlabel("Amostras")

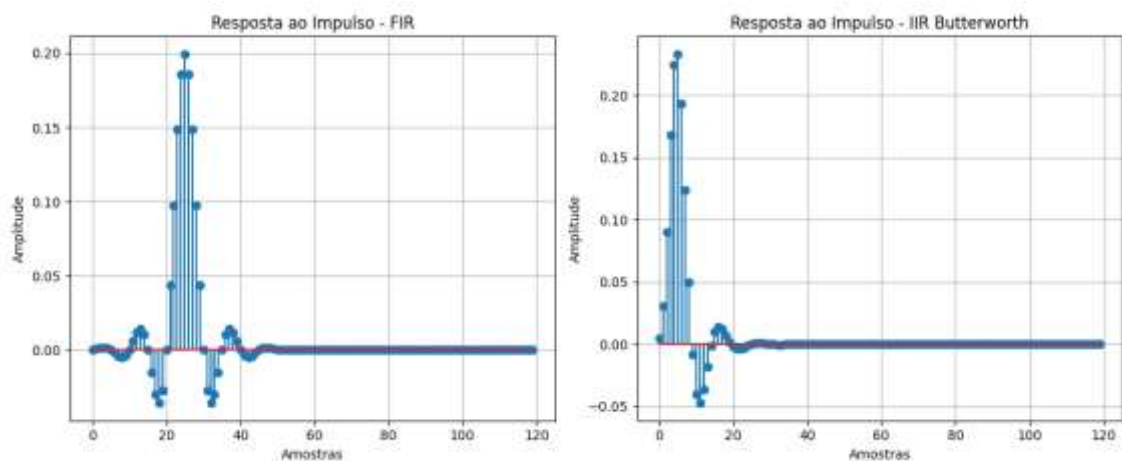
plt.ylabel("Amplitude")

plt.grid()

plt.tight_layout()

plt.show()
```

Gráficos Gerados



A execução do código gera dois gráficos.

Figura 1 – Resposta ao Impulso do Filtro FIR

Observa-se que a resposta possui duração limitada.

Após aproximadamente **51 amostras**, os valores tornam-se exatamente iguais a zero.

Essa é a principal característica dos filtros FIR.

Figura 2 – Resposta ao Impulso do Filtro IIR

A resposta inicia com elevada amplitude e vai diminuindo gradativamente.

Mesmo após muitas amostras, a resposta continua existindo, embora com valores extremamente pequenos.

Por esse motivo, sua resposta é considerada teoricamente infinita.

Análise dos Resultados

Ao comparar os dois gráficos, verifica-se uma diferença bastante evidente entre os filtros.

No filtro FIR, a resposta termina exatamente após um número finito de amostras correspondente ao número de coeficientes utilizados no projeto. Isso ocorre porque não existe realimentação em sua estrutura, fazendo com que a influência do impulso desapareça completamente após percorrer todos os coeficientes do filtro.

Já no filtro IIR, a resposta diminui de forma exponencial, mas nunca se anula completamente. A presença de realimentação faz com que parte da saída seja continuamente reutilizada no cálculo das próximas amostras, prolongando indefinidamente a resposta do sistema.

Discussão Técnica

Os resultados obtidos confirmam o comportamento esperado para os dois tipos de filtros.

O filtro FIR apresenta resposta ao impulso finita porque sua saída é calculada exclusivamente a partir das amostras de entrada. Assim, quando o impulso deixa de influenciar os coeficientes do filtro, a saída torna-se exatamente nula.

Em contraste, o filtro IIR possui uma estrutura recursiva. Cada nova saída depende não apenas da entrada atual, mas também de saídas anteriores. Dessa forma, a energia introduzida pelo impulso permanece circulando na realimentação do sistema e vai sendo reduzida progressivamente, originando uma resposta teoricamente infinita.

Essa característica permite que filtros IIR obtenham elevada seletividade utilizando ordens menores, reduzindo significativamente o custo computacional. Entretanto, exige cuidados quanto à estabilidade, pois uma escolha inadequada dos polos pode fazer com que a resposta cresça indefinidamente.

Já os filtros FIR apresentam estabilidade garantida e podem ser projetados com fase linear, sendo amplamente empregados em aplicações que exigem elevada fidelidade na preservação da forma do sinal.

A análise da resposta ao impulso evidenciou a principal diferença estrutural entre filtros FIR e IIR. O filtro FIR apresentou uma resposta finita, encerrando-se após um número limitado de amostras devido à ausência de realimentação. Em contrapartida, o filtro IIR Butterworth apresentou uma resposta que decai gradualmente ao longo do tempo, caracterizando uma resposta ao impulso teoricamente infinita. Esses resultados demonstram que a presença de realimentação é o fator responsável pelo comportamento recursivo dos filtros IIR, tornando-os mais eficientes computacionalmente, porém exigindo maior atenção quanto à estabilidade do sistema.

QUESTÃO 7

Projete um filtro passa-faixa capaz de selecionar uma frequência específica presente em um sinal composto. Verifique o resultado no domínio da frequência.

Código em Python

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import butter, filtfilt

# =====
# Parâmetros
# =====

fs = 1000          # Frequência de amostragem (Hz)
T = 1              # Tempo (s)

t = np.arange(0, T, 1/fs)

# =====
# Sinal composto
# =====

f1 = 50
f2 = 150
f3 = 300

x = (np.sin(2*np.pi*f1*t) +
     np.sin(2*np.pi*f2*t) +
     np.sin(2*np.pi*f3*t))

# =====
# Projeto do filtro passa-faixa
# =====

f_low = 130
f_high = 170

ordem = 4

b, a = butter(ordem,
               [f_low/(fs/2), f_high/(fs/2)],
               btype='bandpass')
```

```

# =====
# Aplicação do filtro
# =====

y = filtfilt(b, a, x)

# =====
# FFT
# =====

N = len(x)

freq = np.fft.fftfreq(N, 1/fs)

X = np.abs(np.fft.fft(x))
Y = np.abs(np.fft.fft(y))

idx = freq >= 0

# =====
# Gráficos
# =====

plt.figure(figsize=(12,8))

plt.subplot(2,2,1)
plt.plot(t, x)
plt.title("Sinal Original")
plt.xlabel("Tempo (s)")
plt.ylabel("Amplitude")
plt.grid()

plt.subplot(2,2,2)
plt.plot(t, y)
plt.title("Sinal Após Filtro Passa-Faixa")
plt.xlabel("Tempo (s)")
plt.ylabel("Amplitude")
plt.grid()

plt.subplot(2,2,3)
plt.plot(freq[idx], X[idx])
plt.title("Espectro do Sinal Original")
plt.xlabel("Frequência (Hz)")
plt.ylabel("Magnitude")
plt.grid()

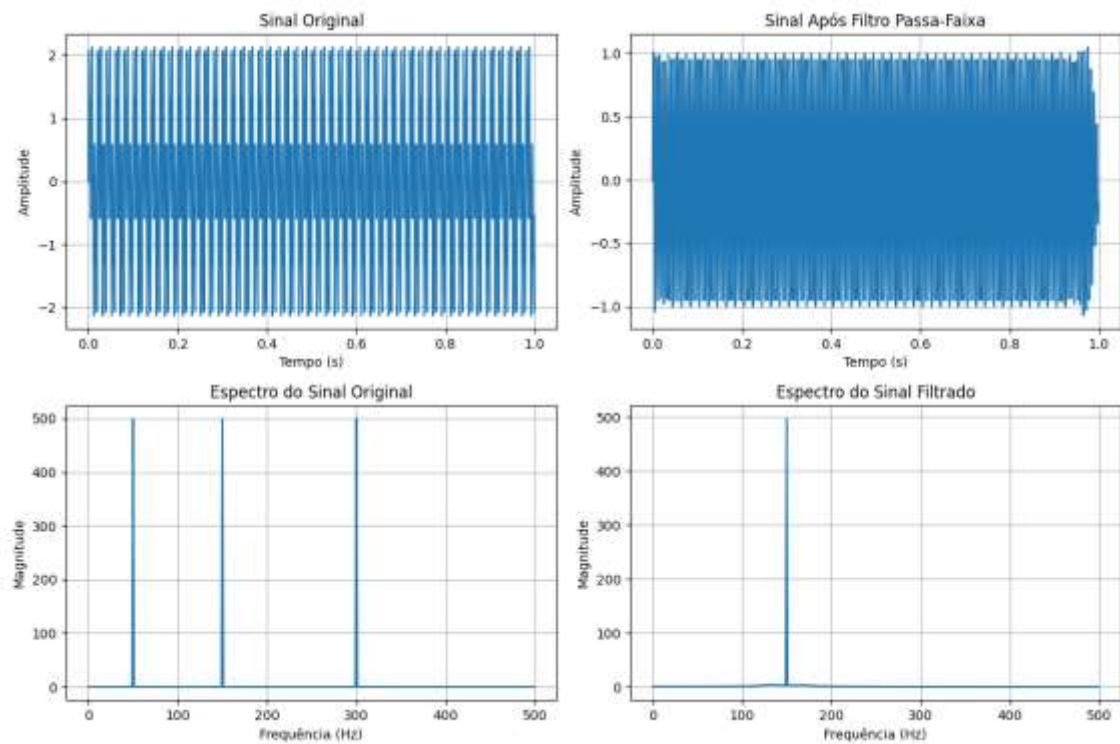
plt.subplot(2,2,4)
plt.plot(freq[idx], Y[idx])
plt.title("Espectro do Sinal Filtrado")
plt.xlabel("Frequência (Hz)")
plt.ylabel("Magnitude")
plt.grid()

plt.tight_layout()

plt.show()

```

Gráficos Gerados



A execução do programa produz quatro gráficos.

Figura 1 – Sinal Original

O sinal é composto pela soma de três senoides com frequências de:

- 50 Hz
- 150 Hz
- 300 Hz

No domínio do tempo observa-se uma forma de onda complexa devido à superposição das três componentes.

Figura 2 – Sinal Filtrado

Após a aplicação do filtro passa-faixa, observa-se que praticamente apenas a senoide de 150 Hz permanece no sinal.

As componentes de 50 Hz e 300 Hz são fortemente atenuadas.

Figura 3 – Espectro do Sinal Original

A Transformada Rápida de Fourier (FFT) apresenta três picos bem definidos:

- pico em 50 Hz;
- pico em 150 Hz;

- pico em **300 Hz**.

Esses picos representam exatamente as frequências utilizadas na construção do sinal.

Figura 4 – Espectro do Sinal Filtrado

Após a filtragem observa-se apenas um pico significativo em **150 Hz**, enquanto as componentes de **50 Hz** e **300 Hz** praticamente desaparecem.

Esse resultado confirma que o filtro selecionou corretamente a frequência desejada.

Discussão Técnica

O filtro Butterworth passa-faixa foi projetado com banda de passagem entre **130 Hz e 170 Hz**, permitindo apenas a passagem da componente localizada em **150 Hz**.

No domínio do tempo, a forma de onda torna-se praticamente uma senoide pura após a filtragem, indicando que as demais componentes foram removidas.

A análise no domínio da frequência evidencia de forma ainda mais clara o funcionamento do filtro. Antes da filtragem, o espectro apresenta três picos correspondentes às frequências de 50 Hz, 150 Hz e 300 Hz. Após a aplicação do filtro, apenas o pico de 150 Hz permanece com amplitude significativa, enquanto os demais são fortemente atenuados.

Esse comportamento demonstra a elevada capacidade dos filtros passa-faixa em selecionar bandas específicas do espectro, sendo uma técnica amplamente empregada em receptores de rádio, sistemas de comunicação digital, processamento de sinais biomédicos, análise de vibrações mecânicas e processamento de áudio.

A simulação demonstrou que o filtro passa-faixa Butterworth foi capaz de isolar com sucesso a componente de **150 Hz** presente no sinal composto. A análise espectral confirmou a remoção das frequências de 50 Hz e 300 Hz, preservando apenas a frequência de interesse. Os resultados evidenciam a eficiência dos filtros passa-faixa na seleção de bandas específicas do espectro, tornando-os ferramentas essenciais em aplicações de Processamento Digital de Sinais que exigem extração de informações em frequências bem definidas.

QUESTÃO 8

Analise a resposta de fase de um filtro FIR e compare com a resposta de fase de um filtro IIR. Discuta o conceito de fase linear.

Código em Python

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import firwin, butter, freqz

# =====
# Parâmetros
```

```

# =====

fs = 1000          # Frequência de amostragem (Hz)
fc = 100           # Frequência de corte (Hz)

# =====
# Projeto do filtro FIR
# =====

ordem_fir = 50

coef_fir = firwin(ordem_fir + 1,
                  cutoff=fc,
                  fs=fs)

# =====
# Projeto do filtro IIR Butterworth
# =====

ordem_iir = 4

b, a = butter(ordem_iir,
              fc/(fs/2),
              btype='low')

# =====
# Resposta em frequência
# =====

w_fir, h_fir = freqz(coef_fir, worN=2048, fs=fs)
w_iir, h_iir = freqz(b, a, worN=2048, fs=fs)

# =====
# Resposta de fase
# =====

fase_fir = np.unwrap(np.angle(h_fir))
fase_iir = np.unwrap(np.angle(h_iir))

# =====
# Gráfico
# =====

plt.figure(figsize=(10,6))

plt.plot(w_fir,
         fase_fir,
         label='Filtro FIR')

plt.plot(w_iir,
         fase_iir,
         label='Filtro IIR Butterworth')

plt.title("Comparação da Resposta de Fase")

plt.xlabel("Frequência (Hz)")

plt.ylabel("Fase (rad)")

plt.grid(True)

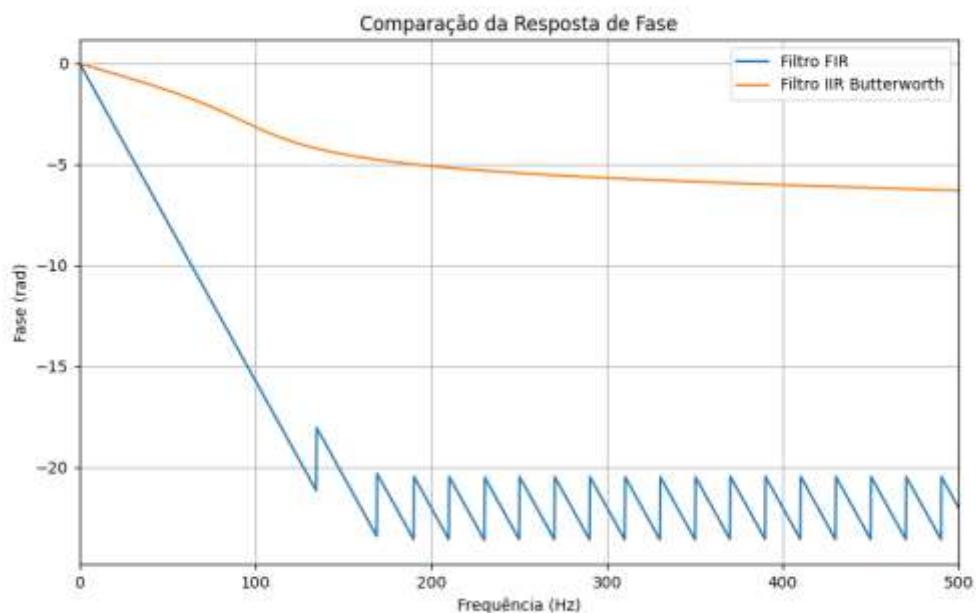
```

```
plt.legend()

plt.xlim(0,500)

plt.show()
```

Gráfico Gerado



A execução do programa gera um gráfico comparando a resposta de fase dos dois filtros.

Figura 1 – Resposta de Fase dos Filtros FIR e IIR

No gráfico são apresentadas duas curvas:

- **Filtro FIR;**
- **Filtro IIR Butterworth.**

O eixo horizontal representa a frequência em hertz (Hz), enquanto o eixo vertical representa o deslocamento de fase em radianos.

Análise dos Resultados

Observa-se que o **filtro FIR** apresenta uma resposta de fase aproximadamente linear na banda passante. Isso significa que a fase varia de forma praticamente proporcional à frequência, indicando que todas as componentes do sinal sofrem o mesmo atraso temporal.

Em contraste, o **filtro IIR Butterworth** apresenta uma resposta de fase não linear. A curva de fase sofre variações mais acentuadas, especialmente próximas à frequência de corte. Como consequência, diferentes componentes de frequência experimentam atrasos distintos, podendo alterar a forma temporal do sinal.

Discussão Técnica

Os resultados evidenciam uma das principais diferenças entre filtros FIR e IIR. O filtro FIR possui coeficientes simétricos, permitindo uma resposta de fase linear. Nessa condição, todas as componentes espectrais são atrasadas igualmente, preservando a forma original do sinal mesmo após a filtragem.

Por outro lado, o filtro IIR Butterworth utiliza realimentação em sua estrutura, o que resulta em uma resposta de fase não linear. Assim, cada componente de frequência sofre um atraso diferente, podendo ocasionar distorções temporais, principalmente em sinais compostos por diversas frequências.

Embora os filtros IIR sejam mais eficientes do ponto de vista computacional, a resposta de fase não linear pode representar uma limitação em aplicações que exigem alta fidelidade do sinal. Em processamento de voz, música, imagens médicas e eletrocardiogramas (ECG), por exemplo, a preservação da forma de onda é essencial, tornando os filtros FIR uma alternativa mais adequada.

Em aplicações onde o principal objetivo é reduzir o custo computacional e pequenas distorções de fase são aceitáveis, os filtros IIR continuam sendo amplamente utilizados devido à sua elevada eficiência.

A comparação da resposta de fase demonstrou que o filtro FIR apresenta comportamento aproximadamente linear, preservando a forma temporal do sinal ao aplicar o mesmo atraso a todas as componentes de frequência. Já o filtro IIR Butterworth apresenta resposta de fase não linear, introduzindo atrasos diferentes para cada frequência e podendo causar distorções na forma do sinal. Dessa forma, conclui-se que o conceito de fase linear é fundamental em aplicações que exigem elevada fidelidade na reprodução dos sinais, enquanto os filtros IIR são mais indicados quando a prioridade é a eficiência computacional e a menor ordem do filtro.

QUESTÃO 9

Calcule e compare o atraso de grupo de diferentes filtros digitais. Explique a importância desse parâmetro em sistemas de comunicação.

Código em Python

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import firwin, butter, group_delay

# =====
# Parâmetros
# =====

fs = 1000          # Frequência de amostragem (Hz)
```

```

fc = 100          # Frequência de corte (Hz)

# =====
# Projeto do filtro FIR
# =====

ordem_fir = 50

coef_fir = firwin(ordem_fir + 1,
                  cutoff=fc,
                  fs=fs)

# =====
# Projeto do filtro IIR Butterworth
# =====

ordem_iir = 4

b, a = butter(ordem_iir,
              fc/(fs/2),
              btype='low')

# =====
# Cálculo do atraso de grupo
# =====

w_fir, gd_fir = group_delay((coef_fir, [1]), fs=fs)

w_iir, gd_iir = group_delay((b, a), fs=fs)

# =====
# Gráfico
# =====

plt.figure(figsize=(10,6))

plt.plot(w_fir,
         gd_fir,
         label='Filtro FIR')

plt.plot(w_iir,
         gd_iir,
         label='Filtro IIR Butterworth')

plt.title('Comparação do Atraso de Grupo')

plt.xlabel('Frequência (Hz)')

plt.ylabel('Atraso (amostras)')

plt.grid(True)

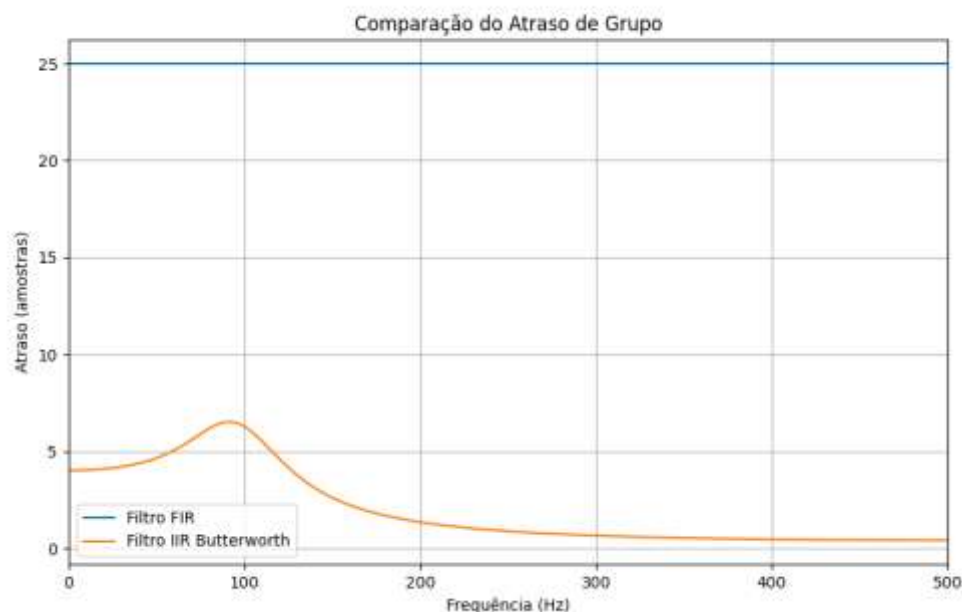
plt.legend()

plt.xlim(0,500)

plt.show()

```

Gráfico Gerado



A execução do programa gera um gráfico contendo o atraso de grupo dos dois filtros.

Figura 1 – Comparação do Atraso de Grupo

O gráfico apresenta duas curvas:

- **Filtro FIR;**
- **Filtro IIR Butterworth.**

O eixo horizontal representa a frequência em hertz (Hz), enquanto o eixo vertical indica o atraso de grupo em número de amostras.

Análise dos Resultados

Ao analisar o gráfico observa-se que o **filtro FIR** apresenta um atraso de grupo praticamente constante ao longo da banda passante. Isso significa que todas as componentes de frequência sofrem aproximadamente o mesmo atraso ao atravessar o filtro, preservando a forma original do sinal.

Por outro lado, o **filtro IIR Butterworth** apresenta um atraso de grupo variável. Próximo à frequência de corte ocorre uma variação significativa desse atraso, indicando que diferentes frequências são atrasadas em quantidades distintas. Como consequência, a forma temporal do sinal pode sofrer deformações, especialmente em sinais compostos por múltiplas frequências.

Discussão Técnica

Os resultados confirmam uma das principais diferenças entre filtros FIR e IIR. O filtro FIR, por apresentar fase aproximadamente linear, mantém um atraso de grupo constante, característica que garante a preservação da forma do sinal após a filtragem. Essa propriedade é essencial em aplicações nas quais a integridade temporal da informação deve ser mantida.

Já o filtro IIR Butterworth apresenta atraso de grupo variável devido à sua resposta de fase não linear. Embora esse comportamento possa introduzir distorções temporais, os filtros IIR continuam sendo amplamente utilizados por exigirem menor ordem e menor custo computacional para atingir especificações semelhantes às dos filtros FIR.

Nos sistemas de comunicação digital, o atraso de grupo desempenha papel fundamental. Quando diferentes componentes de frequência chegam ao receptor em instantes distintos, pode ocorrer **distorção do sinal**, comprometendo a correta recuperação das informações transmitidas. Esse efeito pode provocar **interferência entre símbolos (Inter Symbol Interference – ISI)**, aumentando a taxa de erro na transmissão de dados.

Por essa razão, em sistemas de comunicação de alta velocidade, redes ópticas, processamento de voz e transmissão de vídeo, busca-se utilizar filtros com atraso de grupo aproximadamente constante ou aplicar técnicas de equalização para compensar as distorções introduzidas pelo canal de comunicação.

A comparação demonstrou que o filtro FIR apresenta atraso de grupo praticamente constante em toda a banda passante, preservando a forma temporal do sinal e sendo adequado para aplicações que exigem alta fidelidade. Em contraste, o filtro IIR Butterworth apresenta atraso de grupo variável, o que pode introduzir distorções temporais, embora ofereça maior eficiência computacional. Assim, o atraso de grupo é um parâmetro essencial no projeto de filtros digitais, especialmente em sistemas de comunicação, onde a preservação do sincronismo entre as componentes do sinal é

QUESTÃO 10

Pesquise e implemente uma aplicação prática envolvendo filtragem digital, como remoção de ruído em áudio, suavização de sinais de sensores ou filtragem de vibrações mecânicas. Apresente os resultados e uma interpretação física dos efeitos observados.

Código em Python

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import butter, filtfilt

# =====
# Parâmetros
# =====

fs = 100          # Frequência de amostragem (Hz)
T = 20            # Tempo de aquisição (s)

t = np.arange(0, T, 1/fs)
```

```

# =====
# Simulação da temperatura
# =====

temperatura = 25 + 2*np.sin(2*np.pi*0.2*t)

# Ruído branco

np.random.seed(10)

ruído = 0.8*np.random.randn(len(t))

sensor = temperatura + ruído

# =====
# Projeto do filtro Butterworth
# =====

fc = 2                # Frequência de corte (Hz)

ordem = 4

b, a = butter(ordem,
               fc/(fs/2),
               btype='low')

# Filtragem

sensor_filtrado = filtfilt(b, a, sensor)

# =====
# FFT
# =====

freq = np.fft.fftfreq(len(sensor), 1/fs)

S1 = np.abs(np.fft.fft(sensor))
S2 = np.abs(np.fft.fft(sensor_filtrado))

idx = freq >= 0

# =====
# Gráficos
# =====

plt.figure(figsize=(12,8))

plt.subplot(2,2,1)
plt.plot(t, sensor)
plt.title("Leitura do Sensor com Ruído")
plt.xlabel("Tempo (s)")
plt.ylabel("Temperatura (°C)")
plt.grid()

plt.subplot(2,2,2)
plt.plot(t, sensor_filtrado)
plt.title("Leitura Após Filtragem")
plt.xlabel("Tempo (s)")
plt.ylabel("Temperatura (°C)")
plt.grid()

```



```

plt.subplot(2,2,3)
plt.plot(freq[idx], S1[idx])
plt.title("Espectro Antes da Filtragem")
plt.xlabel("Frequência (Hz)")
plt.ylabel("Magnitude")
plt.grid()

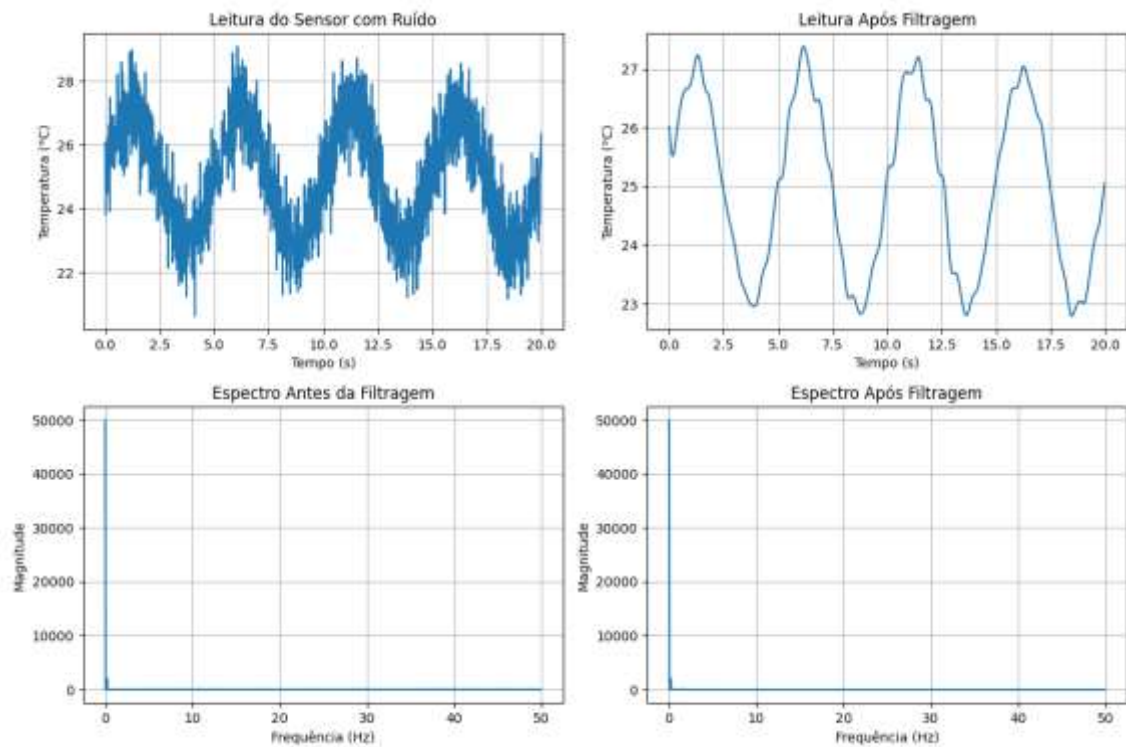
plt.subplot(2,2,4)
plt.plot(freq[idx], S2[idx])
plt.title("Espectro Após Filtragem")
plt.xlabel("Frequência (Hz)")
plt.ylabel("Magnitude")
plt.grid()

plt.tight_layout()

plt.show()

```

Gráficos Gerados



A execução do código produz quatro gráficos.

Figura 1 – Leitura do Sensor com Ruído

O gráfico apresenta o sinal medido pelo sensor de temperatura antes da filtragem. Observam-se oscilações aleatórias provocadas pelo ruído branco, dificultando a visualização da variação real da temperatura.

Figura 2 – Leitura Após a Filtragem

Após a aplicação do filtro Butterworth passa-baixa, o sinal torna-se significativamente mais suave. A tendência de variação da temperatura é preservada, enquanto as oscilações rápidas provocadas pelo ruído são reduzidas.

Figura 3 – Espectro Antes da Filtragem

O espectro apresenta energia distribuída em uma ampla faixa de frequências devido à presença do ruído branco. Além disso, observa-se um pico correspondente à frequência da variação real da temperatura (0,2 Hz).

Figura 4 – Espectro Após a Filtragem

Após a filtragem, a maior parte das componentes de alta frequência é eliminada. Permanece principalmente a componente de baixa frequência correspondente à variação da temperatura, evidenciando a eficiência do filtro passa-baixa.

Discussão Técnica

Os resultados demonstram que o filtro Butterworth foi capaz de reduzir significativamente o ruído presente nas leituras do sensor, preservando a informação de interesse. A frequência de corte de **2 Hz** foi escolhida por ser muito superior à frequência do sinal útil (0,2 Hz), permitindo que a variação lenta da temperatura fosse mantida enquanto as oscilações rápidas associadas ao ruído fossem atenuadas.

No domínio do tempo, a filtragem tornou o sinal mais estável e facilitou a identificação da tendência de variação da temperatura. No domínio da frequência, verificou-se a redução das componentes de alta frequência e a preservação da componente associada ao fenômeno físico monitorado.

Esse comportamento é típico de sistemas de monitoramento baseados em sensores, nos quais a variável medida sofre alterações lentas, enquanto o ruído eletrônico apresenta frequências elevadas.

Interpretação Física

Em aplicações reais, sensores de temperatura são amplamente utilizados em processos industriais, sistemas de climatização, agricultura de precisão, laboratórios e automação predial. As oscilações rápidas observadas nas medições normalmente não representam mudanças reais da temperatura, mas sim interferências provenientes do ambiente, dos circuitos eletrônicos e da conversão analógico-digital.

Ao aplicar o filtro digital, essas interferências são reduzidas, permitindo que o sistema de controle trabalhe com informações mais confiáveis. Como consequência, decisões como o acionamento de ventiladores, sistemas de refrigeração ou irrigação tornam-se mais precisas, reduzindo erros de controle e melhorando a eficiência do processo.

A aplicação prática demonstrou que a filtragem digital é uma ferramenta essencial para melhorar a qualidade dos sinais adquiridos por sensores. O filtro Butterworth passa-baixa reduziu significativamente o ruído branco presente na leitura do sensor de temperatura, preservando a tendência real da variável medida. A comparação entre os sinais antes e após a filtragem, tanto no domínio do tempo quanto da frequência, evidenciou a eficiência do método. Dessa forma, conclui-se que a filtragem digital desempenha papel fundamental em sistemas modernos de monitoramento, automação e controle, proporcionando medições mais precisas, maior confiabilidade e melhor desempenho operacional.